
COMP 4010 - Reinforcement Learning Group 14

Report

Saagar Gahlot
XXXXXXXXXX
REDACTED_EMAIL@XXXXXXXXXX

Stephen Rebel
XXXXXXXXXX
REDACTED_EMAIL@XXXXXXXXXX

Danial McIntyre
XXXXXXXXXX
REDACTED_EMAIL@XXXXXXXXXX

Drax Lindgren
XXXXXXXXXX
REDACTED_EMAIL@XXXXXXXXXX

Sharav Bali
XXXXXXXXXX
REDACTED_EMAIL@XXXXXXXXXX

Abstract

We investigate the use of reinforcement learning for generating 8-bar musical compositions. Our project compares Deep Q-Networks (DQN) and Proximal Policy Optimization (PPO) agents against LSTM and n-gram baselines. We design a multi-component reward function that evaluates rhythm, harmony, progression, and repetition, with penalties to discourage trivial or musically incoherent solutions. Results show that PPO achieves the highest rewards and most coherent compositions, while DQN struggles to learn. Baseline comparisons highlight the difference between RL and supervised learning approaches in capturing long-term musical structure.

1 Introduction

This work explores training a Reinforcement Learning (RL) agent to generate 8-bar musical compositions from scratch. Unlike standard approaches that rely on supervised pre-training, our agent learns purely through interaction with the environment. The core challenge lies in generating coherent sequences guided only by a scalar reward signal. Musical quality is multidimensional and reliant upon aspects such as rhythm, harmony, and structure, among many others. As well, it is highly context-dependent, requiring the agent to balance competing objectives over long time horizons.

This problem offers a valuable testbed for sequential decision-making with sparse, delayed rewards. Unlike typical RL problems with clear objective metrics, music requires defining a reward function for subjective qualities. Investigating how a RL agent learns musicality allows for a direct comparison against traditional deep learning methods and provides insight into long-term dependency problems common in real-world planning systems.

We frame the generation process as a constrained Markov Decision Process (MDP) defined by $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$:

State Space ($\mathcal{S}$): The state s_t represents the musical composition generated up to step t . It is encoded as a fixed-length vector containing the indices of generated notes and the number of beats remaining in the current bar.

Action Space ($\mathcal{A}$): The action space is discrete, consisting of 90 actions. These represent the constrained set of notes spanning two octaves of the C major scale (MIDI pitch 60 to 84), paired with specific note durations (sixteenth to whole notes) and corresponding rests.

Transition Function (P): Transitions are deterministic. When an action is selected, the note is appended to the composition. If a selected duration exceeds the remaining beats in the current bar, it is truncated to fit. The episode terminates when the composition reaches the pre-defined length.

Reward Function (R): We employ a sparse reward structure. Immediate negative penalties are given when invalid actions are selected. The primary reward for a composition is calculated only at the terminal state. It is a mathematical evaluation of rhythmic consistency, harmonic structure, melodic progression, and repetition. Finally penalties are applied for violations of musical properties.

Discount Factor (γ): Given the finite nature of music composition, we do not heavily discount future rewards, setting $\gamma = 0.99$ to incentivize generating valid notes.

2 Approaches

2.1 Prior Works

Traditional methods of music generation often involve supervised learning and deep neural networks like Long short-term memory (LSTM) models [1]. Jaques et al. [2] applied Deep Q-RL [3] fine-tuning to an LSTM with a reward function derived from music theory heuristics (rhythm, harmony, etc.). Their goal was to circumvent the traditional purely data driven approach and generate monophonic melodies. Similarly, in *Bach2Bach*, Kotecha [4] employ a bi-axial LSTM as an initial probabilistic model to which they apply a Deep Q-Learning Network (DQN) RL approach to improve quality.

Other approaches frame music generation as a sequence modelling task, using n-grams [5]. N-grams capture local dependencies between notes or chords but struggle to represent long-term and global structure, motivating the use of reinforcement learning and deep neural networks.

A more recent approach discussed by Cideron et al. [6] employs reinforcement learning from human feedback (RLHF) [7] to a text-to-music auto regressive model. This powerful approach demonstrated significant improvements over their baseline models. *Music Generation using Human-In-The-Loop Reinforcement Learning* [8] is another very recent paper in which the researchers apply RLHF to learn music generation. They implement a tabular Q-learning [9] approach in which human raters provide feedback per generation guiding their model to learn subjective human preferences.

Designing an effective reward function is critical to many RL approaches. Kolic et al. [10] propose methods to mathematically quantify the desirability of rhythm, repetition, motifs, and transitions, computing global quality from local features such as note timings, onsets, durations, and pattern repetitions. Kader & Karmaker [11] also go in depth describing how to measure music generated by AI, and define a clear taxonomy. It breaks down evaluation into many subsections, including Symbolic Score Quality Evaluation which involves using rule-based systems to analyze the structure of a composition and assign it a score, among others.

2.2 General Approach

We evaluate four distinct architectures for music generation. We implement two reinforcement learning agents: Deep Q-Networks (DQN) and Proximal Policy Optimization (PPO), and compare them against N-gram models and LSTM neural networks as baselines. Although all approaches operate within the same constrained environment, each employs different learning methods.

2.3 Reinforcement Learning Agents

Our primary focus lies in training RL agents from scratch to generate musical compositions through environmental interaction. We employ the Stable-Baselines3 library [12] to implement two state-of-the-art RL algorithms.

2.3.1 Deep Q-Network (DQN)

DQN is a value-based off-policy algorithm that learns to approximate the action-value function $Q(s, a)$, representing the expected return when taking action a in state s and following the optimal policy thereafter [3]. The algorithm maintains two key components: a replay buffer storing past experiences (s, a, r, s') and a target network to stabilize training. The Q-network is updated by minimizing the temporal difference error:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q_{\text{target}}(s', a') - Q(s, a; \theta) \right)^2 \right]$$

2.3.2 Proximal Policy Optimization (PPO)

PPO is an on-policy actor-critic algorithm that directly learns a parameterized policy $\pi_{\theta}(a|s)$ while estimating a value function $V(s)$ [13]. Unlike DQN, PPO updates the policy using the clipped surrogate objective to prevent destructively large policy updates:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

where $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio and $\hat{A}_t$ is the advantage estimate computed using Generalized Advantage Estimation (GAE) with $\lambda = 0.95$.

Table 1: Hyperparameters used for the DQN and PPO agents

Hyperparameter	DQN	PPO
Learning rate	5×10^{-5}	3×10^{-4}
Discount factor (γ)	0.99	0.99
Replay buffer size	250,000	–
Target update interval	5000 steps	–
Batch size	128	256
Exploration fraction	0.5	–
n_{steps}	–	1024
n_{epochs}	–	10
GAE λ	–	0.95
Clip range	–	0.2
Entropy coefficient	–	0.015
Value function coefficient	–	0.5
Max gradient norm	–	0.5
Max episodes	10,000	10,000

The hyperparameters we used for both agents are shown in Table 1. These were chosen to maximize our average reward, as well as to produce musically coherent compositions. We use a Multi-Layer Perceptron (MLP) policy network that processes the flattened observation vector (musical score representation plus remaining beats). The agents receive no musical training data and learn purely from the reward signal defined by our multi-faceted reward function.

2.4 Baseline Methods

To contextualize the performance of our RL agents, we implement two supervised learning baselines trained on musical data.

2.4.1 N-gram Language Model

We adapt the n-gram language modelling approach to musical sequence generation using NLTK’s implementation. The model treats musical actions as tokens and learns conditional probability distributions $P(a_t | a_{t-n+1}, \dots, a_{t-1})$ from a corpus of collected compositions [14]. We use a trigram model ($n = 3$) with Laplace smoothing to handle unseen action sequences. The vocabulary consists of all 90 possible actions from our action space plus start and end tokens. The training corpus contains transformed compositions from the Nottingham music dataset [15]. During generation, the

model samples the next action based on the learned conditional probabilities, constrained to select only valid actions given the remaining beats in the current bar. When the model encounters contexts with no learned transitions, it falls back to uniform sampling over valid actions. This approach provides a probabilistic baseline that captures local sequential patterns in musical composition.

2.4.2 LSTM Neural Network

Our LSTM baseline employs a recurrent neural network to model longer-term dependencies in musical sequences [1]. The architecture consists of an embedding layer mapping action indices to dense vectors (128 dimensions), a single-layer LSTM with 256 hidden units and dropout (0.3), followed by a fully-connected output layer producing logits over the action vocabulary. The model is trained on the same action sequence dataset using teacher forcing, where at each time-step the model predicts the next action given the previous ground-truth actions. We use cross-entropy loss with padding token masking:

$$\mathcal{L} = - \sum_{t=1}^T \log P(a_t | a_{1:t-1}; \theta)$$

Due to the variable and often lengthy compositions in the Nottingham MIDI dataset [15], we chunk sequences into overlapping segments of 64 actions to facilitate efficient training. The model trains for 20 epochs using Adam optimizer with learning rate 10^{-3} and weight decay 10^{-4} , saving the best model based on validation loss. For generation, the LSTM auto-regressively samples actions, maintaining hidden state across the composition. Like the n-gram model, action selection is constrained to valid durations given bar boundaries. The LSTM’s recurrent architecture enables it to capture longer melodic and rhythmic patterns compared to the Markov assumption of n-grams.

2.5 Reward Function Design

The reward function serves as the sole learning signal for our RL agents and plays a critical role in shaping compositional quality. We design a multi-component reward function that evaluates four fundamental aspects of musical structure: rhythmic coherence, harmonic correctness, functional chord progression, and repetition of notes and motifs. Each element produces a normalized score in $[0, 1]$. After combining these components with their respective weights, we subtract several global penalties designed to prevent musically nonsensical behaviour. This results in a final scalar reward our agent attempts to maximize.

2.5.1 Rhythm Reward

The rhythm reward evaluates the temporal organization across the sequence of notes. Each note onset is mapped to a grid of four subdivisions per quarter note, allowing rhythms to be compared independently of their tempo. [10] There are five metrics that contribute to this reward component:

Quarter-beat ratio: We compute the fraction of notes beginning on strong beats by dividing the number of onsets divisible by 4 over the total number of onsets. We do this to maintain structure while still allowing syncopation. Notes starting on strong beats give the melody a rhythmic grounding, and deviations from the beat allow for more tension.

Inter-onset interval stability (rhythmic regularity): Measures spacing between notes. We reward consistent pacing using variance of IOIs:

$$s_{\text{IOI}} = 1 - \frac{\text{Var}(t_{i+1} - t_i)}{(\mathbb{E}[t_{i+1} - t_i])^2 + \varepsilon}.$$

Entropy: To prevent monotonous or overly chaotic rhythms, we compute the entropy of note placements within a bar. A moderate entropy value corresponds to the level of rhythmic variety typically seen in simple melodies, whereas very low entropy signifies too much rhythmic repetition, and high entropy signals rhythmic noise.

Note density: We reward melodies whose amount of notes per bar fall near an expected range, preventing overly sparse or overly dense rhythms.

Syncopation: We target moderate syncopation by rewarding off-beat onsets up to a 20% rate, with the intentions of influencing expressive accent patterns without destabilizing the whole rhythm.

The final rhythm score is the weighted average of all five metrics using the tuned weights (1.8, 1.2, 0.9, 0.8, 1.0).

2.5.2 Harmony Reward

To evaluate correctness of the harmony, we assign each bar a chord label using a rule-based detector that scores candidate chords according to three factors: Pitch content, beat salience, and note duration weighting. Given the detected chord for a bar, each note contributes to the harmony score as follows [16]:

- **Chord tones** receive a reward of +2.0, as these tones establish the harmonic identity of the bar and are important to stabilizing the tone.
- **Diatonic non-chord tones (NCTs)** are classified using intervallic relationships and receive fractional credit depending on their melodic function. Passing, neighbour, and escape tones earn +0.5; appoggiaturas +0.75; anticipations +0.6; suspensions and retardations +0.75 and +0.65 respectively. Lastly, pedal tones receive +0.6. Rewarding NCTs prevents the agent from over-fitting to melodies that are completely chordal, resulting in more expressive and human-like melodic patterns.

To prevent chord-heavy bars from negatively influencing the proportions of the total reward, we calculate our total harmony reward as the following:

Let S_{harmony} denote the sum of all per-note contributions. We normalize using an ideal maximum per-note value of 1.7:

$$R_{\text{harmony}} = \frac{1}{1.7} \cdot \frac{S_{\text{harmony}}}{\# \text{melodic notes}}$$

2.5.3 Progression Reward

Harmonies that are of basic correctness are not sufficient enough for generating structured and coherent music, so the progression reward evaluates how well adjacent bars follow harmony motion in appropriate style [17].

The process is as follows: Detect the chord in each bar b : C_b , map each chord to a functional root to normalize chord qualities, and then finally look up the transition weight $w(C_b \rightarrow C_{b+1})$ describing how strong that motion is.

The final progression score averages these transition weights:

$$R_{\text{progression}} = \frac{1}{N_{\text{valid}}} \sum_b w(C_b \rightarrow C_{b+1})$$

where N_{valid} is the number of valid bar pairs. A valid bar pair means that they have detected chords in the pair. Stronger transitions receive higher weights, meaning there is smooth and coherent movement across bars.

2.5.4 Repetition Reward

To achieve motif structure, we flatten the melody into a sequence of (pitch, duration) tokens and compute contiguous n -gram motifs for $n \in \{2, 3, 4\}$. These short fragments model common melodic motifs without requiring explicit boundaries.

For each n we count motif occurrences and compute:

Meaningful repetition: Motifs appearing 2–3 times contribute positively, however motifs that are repeated 4+ times result in penalties to prevent the agent from cheating the reward.

Variety bonus: We reward the presence of multiple distinct motifs that repeat:

$$r_{\text{variety}} = \min\left(0.3 \cdot \frac{|M_{\text{rep}}|}{|M_{\text{all}}|}, 0.2\right).$$

Novelty bonus: Motifs occurring only once also contribute, balancing repetition with interest.

The final repetition reward is capped:

$$R_{\text{repetition}} = \min(r_{\text{motif}} + r_{\text{variety}} + 0.5 r_{\text{novelty}}, 0.75).$$

2.5.5 Penalties

Our training revealed several faults including things like long-note exploitation, empty bars, and mechanical rhythms. Due to these faults, we decided to introduce auxiliary penalties:

- **Bar monotony:** Penalizes uniform durations within a bar.
- **Early bar quality:** Discourages empty or single-note openings.
- **Long-note exploitation:** Penalizes excessive use of whole notes.
- **Duration variety:** Encourages use of multiple rhythmic values.
- **Structural penalties:** Discourage weak endings and empty mid-piece bars.

2.5.6 Final Combination

All reward component scores are normalized to $[0, 1]$ and combined as:

$$R_{\text{final}} = \max(0, [0.45 R_{\text{rhythm}} + 0.22 R_{\text{harmony}} + 0.18 R_{\text{progression}} + 0.15 R_{\text{repetition}}] - \text{Total_Penalty})$$

The weights were selected specifically to reflect the importance of each musical dimension relative to each other, with rhythm emphasized due to its strong influence on how music is perceived.

This design reflects a broader challenge in applying RL to a domain that relies on creativity rather than simple logic. Having to formalize subjective musical criteria into objectives that we could quantify and have remain learnable was difficult to formulate. Our approach to our reward balances the structure of music theory with elements on tuning, to receive coherent and consistent compositions in terms of melody, harmony, rhythm, and expression.

3 Empirical Studies

Each model was trained using multiple random seeds to account for variance in the learning process. Training stability was monitored through episodic rewards, and the final policies were assessed based on the quality of generated sequences. This comparative analysis examines the differences between both the proposed RL agents and the baseline methods.

3.1 Reward Function Validation

We validate our reward function’s calibration by testing it on four well known melodies which were manually encoded in our environment: "Twinkle Twinkle Little Star", "Itsy Bitsy Spider", "Ode to Joy", and "Happy Birthday".

Table 2: The reward scores for known melodies (benchmark testing)

Melody	Final	Base	Rhythm	Harmony	Prog.	Rep.
Twinkle Twinkle	0.190	0.760	0.722	0.683	1.000	0.698
Itsy Bitsy Spider	0.758	0.758	0.709	0.866	0.900	0.578
Ode to Joy	0.000	0.740	0.740	0.825	0.700	0.664
Happy Birthday	0.000	0.746	0.721	0.922	0.800	0.498
Average	0.237	0.751	0.723	0.824	0.850	0.610

We validated our reward function against four well-known melodies manually encoded in our environment, all of which can be seen in Table 2. All melodies achieve strong base rewards (0.74–0.76), confirming that the core scoring components behave as expected. However, three pieces suffer steep penalties that drop their final rewards near zero. The stand-out one, Itsy Bitsy Spider, has a diverse

mix of note lengths, rests, and balanced phrasing, and avoids these penalties. The average final score of 0.237 illustrates the difficulty of the task: the agent must produce coherent music while also being more structurally nuanced than some classic melodies. This confirms that our penalty terms work as intended and prevent trivial exploitation of the reward function.

3.2 Quantitative Results

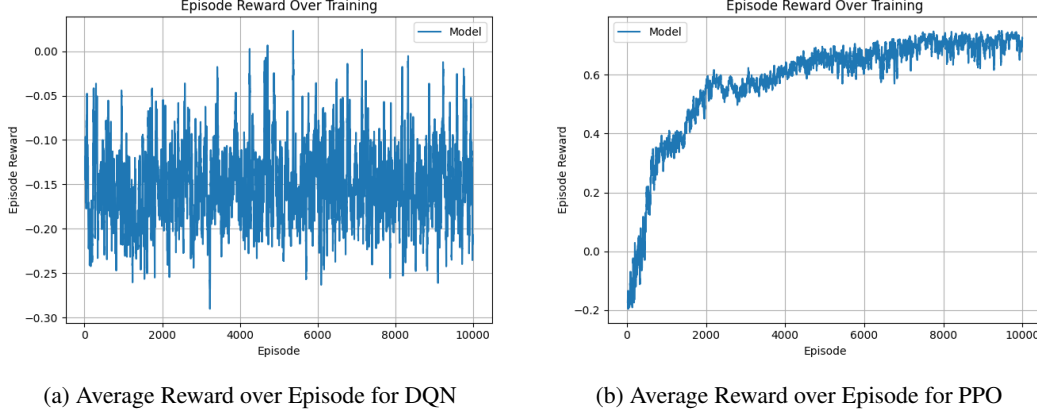


Figure 1: Agent Performance for DQN (left) and PPO (right)

As seen in Figure 1, our DQN agent does not seem to learn at all. Through all the episodes, the average reward per episode ranges from between -0.25 to 0, indicating that there is no learning occurring. There does not seem to be any trend of the average reward as the episodes progress, and there is massive variance in the reward, as it fluctuates greatly between each episode. This is further cemented when looking at the pitch and duration usage histogram of the final policy.

In Figure 2, we can see that the final policy is still uniformly random. Every single note and duration has about the same usage rate, signifying that the agent has not learned anything at all.

In contrast, our PPO agent has a very nice reward curve. In the first 2000 episodes, it spikes to about 0.6, then during the next 8000 episodes, it starts to converge around 0.7. This demonstrates the PPO has found a successful policy capable of achieving a consistent high reward.

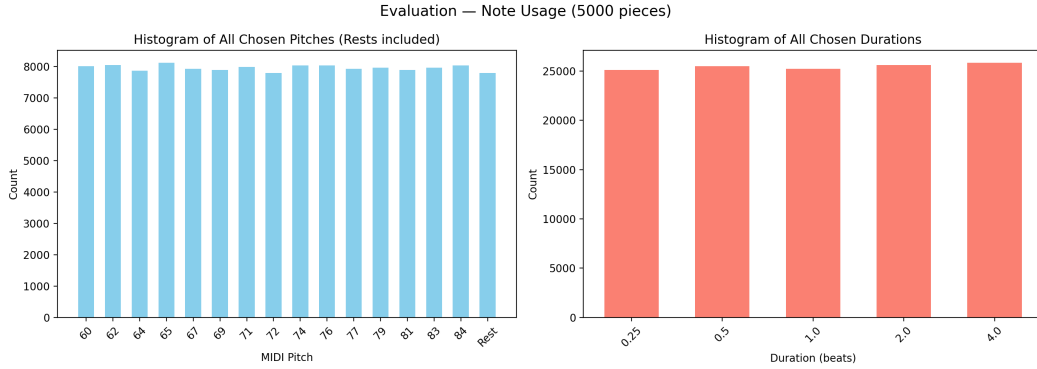


Figure 2: Histogram of pitches and durations for DQN

This is made even more obvious when analyzing the note usage histograms, as seen in Figure 3. We can see the specifics of what the agent has learned. We can see that it really favours the notes 79 (G5), 84 (C6) and 77 (F7), as well as the quarter note (1 beat). This shows us that it has successfully learned some distinct features after training, which follows our musical rules as defined in our reward structure.

Table 3 demonstrates that PPO outperforms all other methods, achieving the highest harmonic, rhythmic, and progression scores with the lowest penalties. We believe PPO's stochastic policy and

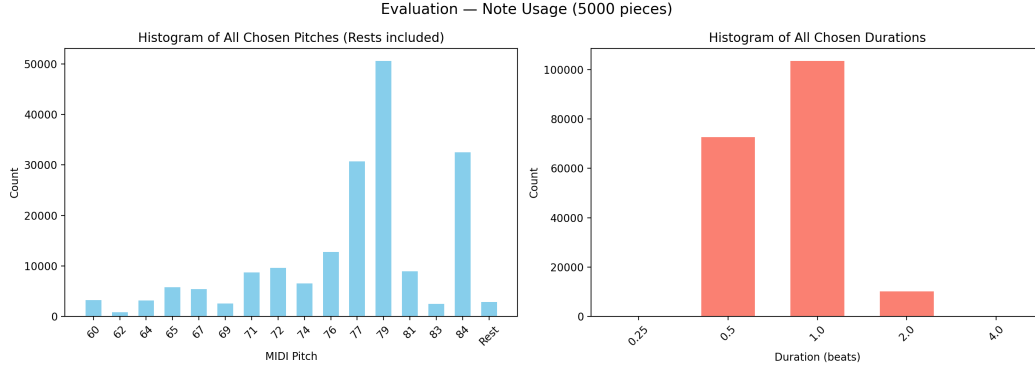


Figure 3: Histogram of pitches and durations for PPO

Table 3: Average composition reward achieved over 5000-trials for differing modelling approaches

Reward Component	PPO	DQN	LSTM	N-Gram
Average final reward	0.7148	0.1115	0.4722	0.4146
Average base reward	0.7391	0.6313	0.6563	0.6592
Average reward penalty	-0.0243	-0.5198	-0.1841	-0.2446
Average rhythm reward	0.7458	0.6028	0.6520	0.6737
Average harmony reward	0.9682	0.9129	0.8836	0.8669
Average progression reward	0.6410	0.4679	0.5180	0.5011
Average repetition reward	0.5010	0.5000	0.5022	0.5007

update stability is the cause, which prevent the failure to learn and repetitive loops observed in the greedy DQN agent. Among the baselines, the LSTM surpasses the N-gram model primarily through reduced penalties, despite similar base rewards (≈ 0.66). Notably, all methods struggle with the repetition metric (≤ 0.5), suggesting a computational ceiling in the reward definition. Furthermore, the poor progression scores for non-RL baselines imply a mismatch between the supervised dataset patterns and the reward function’s criteria.

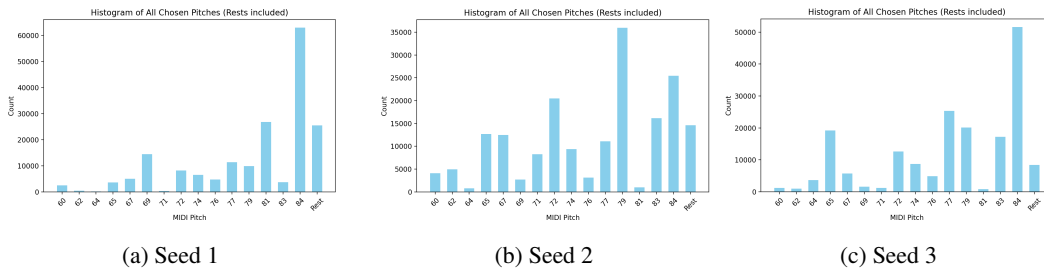


Figure 4: Different pitch distribution based on different seeds

We examined how the learned policy varies across different training runs to see whether the agent would converge to a single “perfect” solution and generate the same piece each time. As shown in Figure 4, runs initialized with three different seeds produce noticeably different pitch distributions, leading to distinct musical outputs. In contrast, the duration distribution remains stable across seeds, suggesting that the agent reliably discovers an optimal duration strategy but not a fixed pitch strategy.

3.3 Qualitative Results

As illustrated in Figures 5a through 5d, the PPO, LSTM, and N-gram models successfully learn fundamental rhythmic and repetitive structures, whereas the DQN agent fails to converge. PPO proves



Figure 5: Comparison of sample scores across different generation models.

to be the superior RL method, producing compositions with distinct motifs and coherent harmonic progressions. In contrast, DQN outputs resemble random noise and lack musical structure.

Among the baselines, the LSTM (Figure 5c) demonstrates strong motifs and harmonic consistency, despite limited diversity. Conversely, the N-gram model yields more diverse outputs but suffers from weaker harmonic coherence. These qualitative trends were observed consistently across sample generations.

Table 4: Average reward for PPO Agent tested with different hyperparameters

Configuration	γ	GAE λ	ent_coef	n_steps	Final Reward
Baseline	0.99	0.95	0.015	1024	0.7148 ± 0.08
Set A	1.0	1.0	0.015	1024	0.7086 ± 0.04
Set B	1.0	1.0	0.0	1024	0.7278 ± 0.03
Set C	1.0	1.0	0.03	1024	0.7066 ± 0.06
Set D	1.0	1.0	0.015	512	0.4985 ± 0.21

We tested the PPO agent against different hyperparameter configurations, as shown in Table 4. In most of the sets, the final average reward varies slightly, however, when looking at the actual music compositions, there are pretty big differences. For example, set A prefers using a lot of 8th notes, and sounds rhythmic pleasing. In contrast, Set B only uses the same 4 notes in a very repetitive and monotone way. Even though Set B achieves a higher average reward, Set A is more musically pleasant to the ear.

4 Conclusion

4.1 Method Outcomes

We have achieved some success in designing an environment and reward function capable of guiding our RL agents to learn a policy for the generation of constrained musical compositions. Our PPO agent succeeds in learning a high reward policy however the quality of the compositions as evaluated by human listeners is moderate. DQN fails entirely to learn. Comparing against supervised baselines, LSTM generation was successful and musically coherent, often exceeding our RL methods as rated by human listeners. We can conclude that the final quality relies heavily upon the reward function and as such its design is crucial to the outcome. It is very challenging to completely capture and describe ideal musical structure across its many dimensions in a purely mathematical formula.

4.2 Future Recommendations

We recommend future research perform a deeper investigation into expressing musical quality mathematically and to focus on actor-critic algorithms like PPO over DQN or other off-policy algorithms. We believe it may be of value to focus reward formulas on particular dimensions of musicality, harmony and rhythm among others, rather than attempting to represent them all as a scalar reward. We are also interested in exploring the use of Reinforcement Learning from Human Feedback for music generation [6], particularly its potential to improve both supervised and unsupervised learning methods, as demonstrated by existing literature.

References

- [1] Hochreiter, S. & Schmidhuber, J. (1997) Long Short-Term Memory. *Neural Computation*, 9(8):1735–1780. 10.1162/neco.1997.9.8.1735
- [2] Jaques, N., Gu, S., Turner, R. E. & Eck, D. (2016) Generating Music by Fine-Tuning Recurrent Neural Networks with Reinforcement Learning. In *Deep Reinforcement Learning Workshop, NIPS 2016*.
<https://research.google.com/pubs/archive/45871.pdf>
- [3] Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D. & Riedmiller, M. (2013) Playing Atari with Deep Reinforcement Learning. *arXiv preprint arXiv:1312.5602*.
<https://arxiv.org/abs/1312.5602>
- [4] Kotecha, N. (2018) Bach2Bach: Generating music using a deep reinforcement learning approach. *arXiv preprint arXiv:1812.01060*. <https://arxiv.org/abs/1812.01060>
- [5] Ogihara, M. & Li, Q. (2008) Composer Identification Using N-Gram Chord Profiles. In *Proceedings of the 9th International Society for Music Information Retrieval Conference (ISMIR 2008)*, pp. 55–60.
- [6] Cideron, G., Girgin, S., Verzetti, M., Vincent, D., Kastelic, M., Borsos, Z., McWilliams, B., Ungureanu, V., Bachem, O., Pietquin, O., Geist, M., Hussenot, L., Zeghidour, N. & Agostinelli, A. (2024) MusicRL: Aligning music generation to human preferences. *arXiv preprint arXiv:2402.04229*. <https://arxiv.org/abs/2402.04229>
- [7] Christiano, P., Leike, J., Brown, T. B., Martic, M., Legg, S. & Amodei, D. (2023) Deep Reinforcement Learning from Human Preferences. *arXiv preprint arXiv:1706.03741*. <https://arxiv.org/abs/1706.03741>
- [8] Justus, A. A. (2025) Music generation using human-in-the-loop reinforcement learning. *arXiv preprint arXiv:2501.15304*. <https://arxiv.org/abs/2501.15304>
- [9] Watkins, C.J.C.H. & Dayan, P. (1992) Q-Learning. *Machine Learning*, 8(3–4):279–292. <https://link.springer.com/content/pdf/10.1007/BF00992698.pdf>
- [10] Kolic, B., Rodriguez-Cervantes, M. T., Padilla-Longoria, P. & Knights, F. (2022) Rhythm and form in music: a complex systems approach. *arXiv preprint arXiv:2207.03602*. <https://arxiv.org/abs/2207.03602>
- [11] Kader, F. B. & Karmaker, S. (2025) A Survey on Evaluation Metrics for Music Generation. *arXiv preprint arXiv:2509.00051*. <https://arxiv.org/abs/2509.00051>
- [12] Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M. & Dormann, N. (2021) Stable-Baselines3: Reliable Reinforcement Learning Implementations. *Journal of Machine Learning Research* 22(268):1–8.
- [13] Schulman, J., Wolski, F., Dhariwal, P., Radford, A. & Klimov, O. (2017) Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347*. <https://arxiv.org/abs/1707.06347>
- [14] NLTK Development Team (2025) NLTK Language Modeling — API reference. *NLTK documentation*. <https://www.nltk.org/api/nltk.lm.html>
- [15] Nottingham Music Dataset (2009) Nottingham Music Database: a collection of British and American folk songs (≈ 1200 tunes, ABC/MIDI). http://ifdo.ca/~seymour/nottingham/nottingham_database.zip
- [16] Piston, W. (2011) *Harmony* (5th ed.). New York: W.W. Norton & Company.
- [17] Kostka, S., Payne, D. & Almén, B. (2017) *Tonal Harmony* (8th ed.). New York: McGraw-Hill Education.